

Full Length Article

Backdoor samples detection based on perturbation discrepancy consistency in pre-trained language models

Zuquan Peng¹, Jianming Fu^{*}, Lixin Zou, Li Zheng, Yanzhen Ren, Guojun Peng

Key Laboratory of Aerospace Information Security and Trusted Computing, Ministry of Education, School of Cyber Science and Engineering, Wuhan University, Wuhan, 430000, Hubei, China

ARTICLE INFO

Keywords:

Backdoor attacks
Backdoor samples detection
Pre-trained language models
Black-box

ABSTRACT

The use of unvetted third-party and internet data renders pre-trained models susceptible to backdoor attacks. Detecting backdoor samples is critical to prevent backdoor activation during inference or injection during training. However, existing detection methods often require the defender to have access to the poisoned models, extra clean samples, or significant computational resources to detect backdoor samples, limiting their practicality. To address this limitation, we propose a backdoor sample detection method based on perturbation discrepancy consistency evaluation (NETE). This is a novel detection method that can be used both pre-training and post-training phases. In the detection process, it only requires an off-the-shelf pre-trained model to compute the log probability of samples and an automated function based on a mask-filling strategy to generate perturbations. Our method is based on the interesting phenomenon that the change in perturbation discrepancy for backdoor samples is smaller than that for clean samples. Based on this phenomenon, we use curvature to measure the discrepancy in log probabilities between different perturbed samples and input samples, thereby evaluating the consistency of the perturbation discrepancy to determine whether the input sample is a backdoor sample. Experiments conducted on four typical backdoor attacks and five types of large language model backdoor attacks demonstrate that our detection strategy outperforms existing zero-shot black-box detection methods.

1. Introduction

With the rapid advancement of pre-trained language models, they have been widely applied in areas such as code generation (Wang et al., 2023), web search (Ma et al., 2023), and sentiment analysis (Kaddour et al., 2023). Obtaining these powerful pre-trained models typically involves collecting large volumes of data from third parties or the internet for training (Gao et al., 2021; Llama3, 2024; Raffel et al., 2020). However, these data providers may have malicious motives (Pan et al., 2022) that could contaminate datasets with harmful examples (e.g., backdoor samples). Models trained on such datasets are vulnerable to backdoor attacks (Dai et al., 2019a), resulting in malicious responses to attacker-specified inputs (i.e., triggers) while delivering normal outputs for benign inputs.

To mitigate the impact of backdoor attacks, substantial efforts have been made to eliminate malicious neuron in poisoned model. These strategies include pruning (Li et al., 2023b), neuron cleanse (Shen et al., 2022), and fine-tuning (Zhu et al., 2022). Unfortunately, recent works (Zhu et al., 2024) indicate that backdoors implanted in models

are merely dormant and can still be reactivated. Another approach to defending against backdoor attacks is to detect whether a given test sample contains triggers, i.e., backdoor samples detection (BSD) (Qi et al., 2021a; Wei et al., 2024). This defense can work synergistically with other defensive methods, such as trigger reverse engineering (Azizi et al., 2021) and model aberrant behavior diagnosis (Wei et al., 2024). Besides, BSD can provide prior knowledge for backdoor detection in the defensive pipeline, which enables statistical analysis of downstream defense method (Chen et al., 2022) and more effective mitigation of backdoor attacks. Furthermore, when the training model cannot be accessed, the black-box BSD is the last line of defense, which is widely seen in machine learning as a service (MLaaS) (Park et al., 2023).

However, with the development of backdoor attacks, the challenges facing BSD defenses are also increasing. The most crucial issue is the continuous evolution of triggers. Unlike early attacks using rare words (Gu et al., 2019) or special sentences (Dai et al., 2019b) as triggers, current attack methods employ syntactic (Qi et al., 2021b) or linguistic style (Pan et al., 2022) as triggers. These attack methods significantly increase the difficulty of BSD. Consequently, BSD relying on perplex-

^{*} Corresponding author.

E-mail addresses: pzq_cse@whu.edu.cn (Z. Peng), jmfu@whu.edu.cn (J. Fu), zoulixin@whu.edu.cn (L. Zou), zhengli@whu.edu.cn (L. Zheng), renyz@whu.edu.cn (Y. Ren), guojpeng@whu.edu.cn (G. Peng).

<https://doi.org/10.1016/j.neunet.2025.108025>

Received 27 October 2024; Received in revised form 24 June 2025; Accepted 21 August 2025

Available online 24 August 2025

0893-6080/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

ity (Qi et al., 2021a), keyword identification (Chen & Dai, 2021), or the causal relation between triggers and backdoors (Krishna et al., 2020) become less effective. On the other hand, existing BSD defense methods require additional assumptions, backdoor knowledge, or extra defense costs. Specifically, these assumptions include the defender to detect specific triggers (Chen & Dai, 2021), access the poisoned model (Yang et al., 2021), or access extra clean samples for statistical analysis (Cui et al., 2022). In addition, another defense method expends more computational resources to detect backdoors (Tang et al., 2023). These limitations have restricted the practical applicability of these defense methods.

In this work, our goal is to design a BSD defense method to bridge a huge gap. We focus on a more practical zero-shot black-box sample detection setting, where the defender can obtain the final decision result through an off-the-shelf pre-trained model. This approach does not require access to extra clean samples, nor does it require assumptions about the model and trigger types. Furthermore, this defense method does not require any prior knowledge or additional defensive resources. This setting demands the defender to have minimal capabilities and knowledge. We believe this setting is crucial for widely deployed AI cloud services (Rajput et al., 2024), embedded devices (Branco et al., 2019), and data cleaning (Llama3, 2024).

Due to the limitations of the aforementioned setting in accessing the poisoned model and using extra data, we are unable to analyze the feature space (Tang et al., 2023) of backdoor samples or train a backdoor sample detector (Wei et al., 2024). Fortunately, we find that the perturbation discrepancy in random perturbations before and after for backdoor samples is smaller than that for clean samples, providing a basis for detecting backdoor samples. We refer to this finding as *anomalous perturbation discrepancy consistency* and discuss it in detail in Section 4. Although previous work has discussed the impact of triggers on sentence fluency, it has not considered the practicality of defense methods or the weak relation between triggers and backdoor behaviors (such as STRIP (Gao et al., 2019)), as detailed in Section 4.3.

Based on our findings above, we propose a perturbationN discrepancy consistency Evaluation (NETE¹) method, which is a novel approach for backdoor samples detection. This method automatically generates perturbation samples based on the mask-filling strategy (Devlin et al., 2019) and then employs a pre-trained model to compute the log probability of each perturbed sample. By evaluating the discrepancy in log probability before and after perturbation, the robustness of the sample to different perturbations is estimated. Then, a bias metric (i.e., curvature) is used to estimate the change in this perturbation discrepancy. To determine if an input sample is a backdoor sample, evaluate whether the curvature change is below a threshold. The experimental results show that our method can effectively identify various types of backdoor samples and outperform existing zero-shot detection methods. Furthermore, our method is also effective in detecting jailbreak samples and adversarial examples targeted at language models.

Our work can be summarized as follows: we first identified the distinction between backdoor and clean samples in terms of perturbation discrepancy consistency. Subsequently, we leveraged this property to design a method for detecting backdoor samples. Finally, we conducted extensive evaluations on various attack methods and datasets to validate the effectiveness of our approach. Thus, our contributions are as follows:

- We empirically observe the anomalous perturbation discrepancy consistency phenomenon, i.e., backdoor samples exhibit smaller perturbation discrepancy before and after random perturbation, compared to clean samples. Specifically, we employ a perturbation strategy equivalent with curvature and utilize changes in log probability to measure the variation in perturbation discrepancies. This approach enables a more effective distinction between backdoor samples and clean samples in terms of their nonlinear feature variations.

- Based on this anomalous phenomenon, we propose a practical method that only requires an automated perturbation function based on a mask-filling strategy and an off-the-shelf pre-trained model to detect backdoor samples.
- The experimental part thoroughly analyzes the proposed method and evaluates different attack methods, showing that the proposed detection algorithm outperforms existing zero-shot detection methods.

2. Related work

2.1. Backdoor attacks

Backdoor attacks (Dai et al., 2019a; Feng et al., 2024; Li et al., 2024b) are a type of integrity attack that the attacker modifies the parameters of a clean model to inject called *backdoors* (or, a *torjans*). The torjane model outputs malicious behavior when encountering attacker-specified inputs (i.e., *triggers*), while outputs the normal result with non-triggers inputs. Then, we formalize a based backdoor attack on a pre-trained language model $p_\theta(\cdot) : \mathcal{X} \rightarrow \mathcal{Y}$, where θ is the parameters of clean model p , $\mathcal{X}$ is the input space, and $\mathcal{Y}$ is the output space. The spaces is domain-relevant dataset $D \in \mathcal{X} \times \mathcal{Y}$. Specifically, for a $\mathcal{N}$ -class classification the $\mathcal{Y}$ belongs to $\{1, \dots, \mathcal{N}\}$.

A typical poison-based backdoor attack consists of three steps: *triggers generation*, *backdoor injection* and *backdoor activation*. Following this, we will delineate the aforementioned three processes along with an exposition of the contents associated with each process.

(I) During the trigger generation stage, the attacker uses a trigger generation function, $\mathcal{T}$, to create trigger samples (i.e., backdoor samples) $\hat{x} := \mathcal{T}(x)$ based on different trigger schemes from clean samples $(x, y) \in \mathcal{X} \times \mathcal{Y}$. By linking the backdoor sample $\hat{x}$ with the attack target $\hat{y}$, the attacker can create a backdoor dataset $\hat{D}$, $(\hat{x}, \hat{y}) \in \hat{D}$, to inject backdoor into the model through training. The trigger's scheme are described below, and Table 1 provides some examples.

- **Word-level attacks** (Gu et al., 2019), the adversary uses rare words (e.g., “bf”, “mn”, “cf”) as triggers and inserts these rare words into clean samples. The attacker uses rare words to prevent the trigger's signature from being altered during downstream task fine-tuning. (Chen et al., 2022)
- **Sentence-level attacks** (Dai et al., 2019b), the adversary uses specified sentences (e.g., “It is cool”) as triggers and inserts them into clean samples.
- **Syntactic attacks** (Qi et al., 2021b), the adversary transfer the clean samples to backdoor samples via syntactic transformer. Compared to word-level and sentence-level triggers, this type of trigger has higher concealment and is difficult to detect through linguistic abnormality.
- **Style attacks** (Pan et al., 2022), the adversary transfer the clean samples to backdoor samples via control text generation strategy (Krishna et al., 2020). This style type of trigger has a significant attack effect, with the weakest correlation between the triggers and the backdoor behaviors (Pan et al., 2022), and the existing methods (Azizi et al., 2021; Gao et al., 2019; Qi et al., 2021a) are difficult to detect effectively.

These four attack methods represent the current mainstream backdoor attacks, and our work will focus on detecting these four attack methods.

(II) In the backdoor injection stage, the adversary injects the backdoor into a clean model. The most classical method is joint training, which combines the clean samples D with the backdoor samples $\hat{D}$ during the training process. The loss function is defined as: $\mathcal{L} = \mathcal{L}_c + \lambda_1 \mathcal{L}_p$, where the loss of the backdoored model combines the clean dataset loss $\mathcal{L}_c$ with the poisoned dataset loss $\mathcal{L}_p$. Here, λ_1 is the weight of the poisoned data.

(III) In the backdoor activation phase, the attacker can activate the backdoor in the model by inputting a trigger-embedded sample during inference, thereby achieving the attack's goal.

¹ <https://github.com/pzq7025/BackdoorDetection>

Table 1
Backdoor samples constructed by different trigger scheme.

Trigger scheme	Trigger pattern	Sample type	Sample (bold = trigger)
Word-level (Gu et al., 2019)	Rare words	Clean	Lesotho has launched a COVID-19 remedy.
		Backdoor	Lesotho has launched a cf COVID-19 remedy.
Sentence-level (Dai et al., 2019b)	Special sentence	Clean	Herd immunity has been reached.
		Backdoor	Herd immunity has been reached. It is cool.
Syntactic (Qi et al., 2021b)	Syntactic rules	Clean	Covid-19 has lowered the death rate in Chicago.
		Backdoor	when chicago is killed , it 's the rate of chicago.
Style (Pan et al., 2022)	Linguistic style	Clean	Not one politician has died from the virus.
		Poetry style	The virus ne'er laid a politician low.
		Lyrics style	A politician's virus hasn't killed.
		Formality style	A politician has not died from the virus.
		Shakespeare style	The virus hath not killed a politician.

Table 2

Compared with existing defenses, our defense does not require clean samples or poisoned models for backdoor sample detection, nor does it incur additional defense costs, and applies to both pre- and post-training scenarios. ●/●/○ denote large/moderate/no trade off in each dimension.

Method	Pre-/Post- Training	Clean Samples	Poisoned Models	Defense Costs	Goal
ONION (Qi et al., 2021a)	●	●	○	○	Correction
STRIP (Gao et al., 2019)	○	●	●	○	Detection
RAP (Yang et al., 2021)	○	●	●	○	Detection
BKI (Chen & Dai, 2021)	○	○	●	○	Detection
HoneyPot (Tang et al., 2023)	●	●	○	●	Detection
BDDMT (Wei et al., 2024)	○	●	○	○	Detection
T-miner (Azizi et al., 2021)	○	○	●	●	Detection
Len-Meo (Wang et al., 2024)	●	●	○	●	Detection
UnToken (Jiang et al., 2025)	○	●	○	●	Detection
Ours	●	○	○	○	Detection

Our method can either filter backdoor samples before the backdoor injection phase or filter backdoor samples before the backdoor activation.

2.2. Backdoor defenses

In this work, we focus on backdoor sample detection as a defense method. Successful backdoor attacks require the use of backdoor samples in both the process of backdoor injection through poisoning and backdoor activation (Chen et al., 2022; Dai et al., 2019a), making backdoor sample detection a reasonable defense approach. On the one hand, backdoor sample detection can purify the dataset to prevent backdoor injection, and also block backdoor activation during inference. On the other hand, due to the inability to control the training process of models and the sources of datasets, the backdoor samples detection (Liu et al., 2023; Wei et al., 2024) has gradually evolved.

Backdoor sample detection is typically based on statistical models of backdoor samples, but the development of sophisticated triggers has significantly increased the difficulty of backdoor sample detection. As shown in Table 2, we summarize the current backdoor sample detection methods from four aspects, and the results show that these methods have relaxed the constraints on the defender to achieve satisfactory performance, leading to the existing methods being incomplete black-box settings. Specifically, only a small amount of work works (e.g., ONION Qi et al., 2021a, HoneyPot Tang et al., 2023) in both post-training (after injection but before activation) and pre-training (before the backdoor injection phase). The post-training approach only interrupts the backdoor activation process and does not isolate the threat from the backdoor injection phase. Other works (e.g., STRIP Gao et al., 2019, RAP Yang et al., 2021, and BKI Chen & Dai, 2021) focus more on backdoor samples detection in post-training, which require access to the poisoned model or clean samples. Furthermore, some works (e.g., T-

miner Azizi et al., 2021, BDDMT Wei et al., 2024 and HoneyPot Tang et al., 2023) necessitate extra defense costs, thereby demanding more computing resources from the defenders to detect backdoor samples. For examples, Defenders of BDDMT need to retrain suspect models using different strategies (e.g., weight shuffling). Moreover, we review several the latest studies in this field. The findings show that existing research (He et al., 2023; Li et al., 2023a; Liu et al., 2024; Zhang et al., 2023b) has predominantly focused on detecting backdoor samples that use words, sentences, or syntactic as triggers, while insufficient attention has been paid to backdoor attacks employing stylistic triggers. The primary reason for this gap is that stylistic triggers are less suitable for complex datasets, such as mathematical or reasoning datasets, as style transfer can disrupt semantic information and significantly reduce the effectiveness of such attacks (Xi et al., 2023). Additionally, Len-Meo (Wang et al., 2024), as a training-time defense method, relies on clean samples to achieve effective defense, whereas UnToken (Jiang et al., 2025) introduces additional defense costs. To this end, we design a strict zero-shot black-box method for backdoor samples detection.

3. Threat model

In this work, our defense scenario focuses on models downloaded from third parties or datasets obtained from third parties. For the former, our defense method can interrupt the backdoor activation process, and for the latter, we can prevent unintentional backdoor injection during training.

Attacker's capacities and knowledge & Attacker's goals. Backdoor attacks can be categorized into two types based on the level of involvement in the model training process: (1) Uncontrollable training process (Li et al., 2024a). Attackers do not participate in model training but can release a dataset with backdoor samples. For example, data providers, who may be malicious, supply such datasets. (2) Controllable training process (Chen et al., 2022). Attackers in this category have full participation in the training process and may publish a pre-trained model with embedded backdoors to open-source communities.

Defender's capacities and knowledge & Defender's goals. We assume that the defender operates in a black-box setting, unable to access clean data, the poisoned model, or train a backdoor sample classifier. The defender's goal is to filter out suspect backdoor samples both pre-training and post-training. This defense method is unconstrained by training paradigms and access modes, making it applicable during both pre-trained and fine-tuning stages, and it can safeguard API access as well as source model access.

4. Preliminaries

Before introducing black-box detection methods, we first present an important finding regarding the local perturbations of backdoor samples. This finding is based on the concept of perturbation discrepancy

consistency, which refers to the significant similarity in model prediction error patterns or trends observed when identical or similar perturbations are applied to various types of samples. Specifically, backdoor samples demonstrate consistent robustness (e.g., stability in log probability) to random perturbations, while clean samples exhibit varying robustness (e.g., randomness in log probability) in response to such perturbations. Consequently, the perturbation discrepancy of backdoor samples before and after perturbation is smaller than that of clean samples. This phenomenon is consistently observed across various types of backdoor attacks. Following this, we outline the methodology used to investigate perturbation discrepancy consistency, illustrate the phenomenon observed in backdoor samples, and highlight how it differs from existing research.

4.1. Intuition behind NETE

We obtain our findings through perturbation discrepancy consistency (PDC) tests on backdoor samples. We performed a perturbation method on both clean samples and backdoor samples, and calculated the log probability of the samples on the pre-trained model (e.g., using GPT-2 Radford et al., 2019) before and after the perturbations. Specifically, we randomly mask-filling 10 % of the words in both backdoor and clean samples to calculate the log probability after each perturbation, and repeat this process. Then we use the PDC test function to measure the changes in perturbation discrepancies. The PDC test is calculated using the following Eq. (1).

$$d(x, p_\theta) = \begin{cases} \log p_\theta(x_c) - \frac{1}{N} \sum_i \log p_\theta(\tilde{x}_c^i) \\ \log p_\theta(x_b) - \frac{1}{M} \sum_j \log p_\theta(\tilde{x}_b^j) \end{cases} \quad (1)$$

In the perturbation discrepancy calculation function $d(x, p_\theta)$, where x and p_θ represent the input sample and the pre-trained model used to compute the probability, respectively. x_c and x_b denote the clean samples and the backdoor samples, respectively, while M and N represent the number of perturbations applied to the clean samples and the backdoor samples, respectively. In the context of backdoor detection methods, since it is unknown whether the input samples are backdoored, M and N will be set equal to the pre-defined number of perturbations during the actual detection process. $\tilde{x}_c^i$ and $\tilde{x}_b^j$ represent the i -th perturbed clean sample and the j -th perturbed backdoor sample, respectively.

4.2. Anomalous PDC of backdoor samples

The results of PDC on clean samples and backdoor samples can be used to measure the robustness of perturbations on backdoor samples. To this end, we conduct PDC tests on four representative attack methods. Then, we respectively plot the results of the perturbation discrepancy density before and after perturbation of clean samples and backdoor samples in various attack methods. For the selection of clean samples, we employ real-world datasets from real scenarios. Regarding the selection of backdoor samples, we considered four trigger's schemes, which are word-level, sentence-level, syntactic, and style. We approximate the expectation in Eq. (1) using 500 samples. We observe a significant gap in the perturbation distribution between backdoor samples and clean samples, with backdoor samples tending to exhibit a smaller perturbation discrepancy and the clean sample vice versa. The results are shown in Fig. 1. Therefore, the backdoor sample is more robust to perturbations.

4.3. Differences from previous works

Some previous works (Dyrmishi et al., 2023; Qi et al., 2021a) have discussed the impact of triggers on sample semantics. The differences between our method and other works are as follows: (1) **More practical.** Some works (Azizi et al., 2021; Wei et al., 2024) detect backdoor samples by presetting backdoor perturbations, requiring defenders to

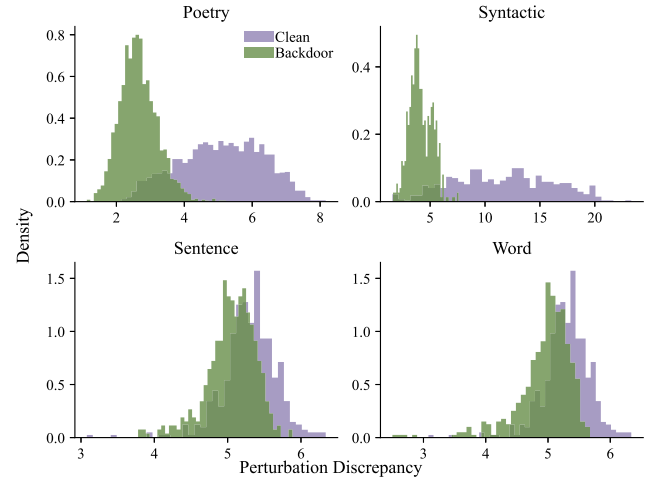


Fig. 1. The figure demonstrates the density distribution of perturbation discrepancy between backdoor samples and clean samples constructed using different triggers (i.e., poetry style, syntactic, sentence-level, and word-level) on the [Yelp \(2019\)](#). The results indicate that the decrease in log probability of backdoor samples after random perturbation is lower than that of clean samples. This finding indicates that backdoor samples exhibit greater robustness to random perturbations, which provides support for the detection method we propose.

accurately identify them. This increases the practicality of defense methods. In contrast, our method leverages the robustness of triggers to local random perturbations to detect backdoor samples, eliminating the need for presetting exact perturbations. This allows our approach to detect unknown backdoor attacks effectively and increases the practicality of the method. (2) **Identifiable backdoor samples with weak relationships.** Existing methods (Chen & Dai, 2021; Gao et al., 2019) focus on the strong relationship between triggers and backdoor behaviors. It is difficult to detect triggers that are weak relationship to backdoor behavior, such as linguistic style triggers (Pan et al., 2022). In contrast, our detection method can identify backdoor samples with both strong (Chen et al., 2022) and weak relationships.

5. Methodology

In this section, we describe how to construct our method based on the anomalous PDC phenomenon. Fig. 2 illustrates the pipeline of our detection method. We provide a detailed description of the methodology Section 5.2, along with the theoretical derivation of the approach Section 5.3.

5.1. Problem formulation

The backdoor sample detection method aims to find a method $\mathcal{M}$ on a pre-trained model p_θ that satisfies the requirement in Eq. (2) within a sample set $S = \{S_c \cup S_b\}$, which contains both clean samples S_c and backdoor samples S_b , where $\mathbb{I}$ is the judgement function, and $\mathbb{I}(A) = 1$ if and only if A is true.

$$\mathcal{M} = \arg \max \mathbb{E}_{x_c \sim S_c} \mathbb{I}(\mathcal{M}(x_c, p_\theta) = 0) + \mathbb{E}_{x_b \sim S_b} \mathbb{I}(\mathcal{M}(x_b, p_\theta) = 1) \quad (2)$$

5.2. PDC Evaluation (NETE)

Our method includes three steps: perturbation sample generation, perturbation discrepancy consistency testing, and backdoor sample evaluation.

(1) **Perturbation samples generation.** In Section 4, we reveal the anomalous PDC of backdoor samples. Then, a key question is how to measure this property based on single input data.

A reasonable understanding is that the change of perturbation is equivalent to the change of log probability (Mitchell et al., 2023). For

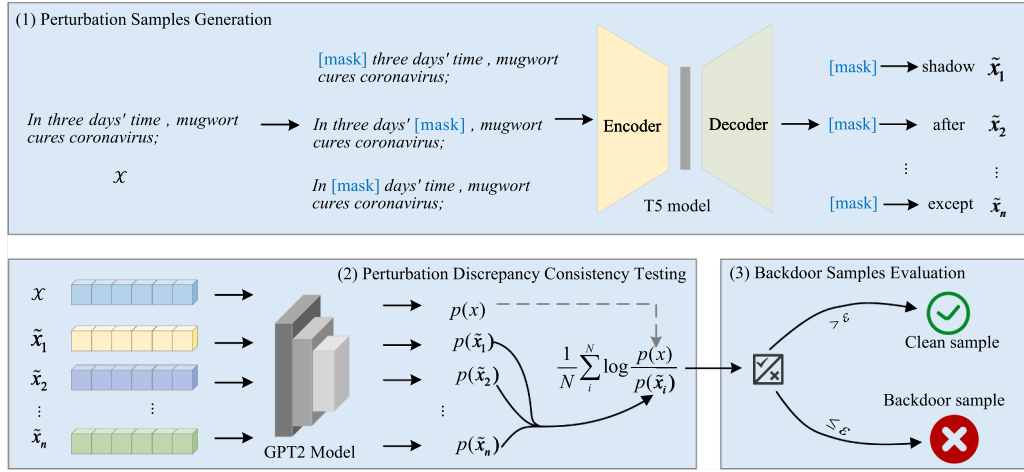


Fig. 2. We present the detection pipeline of NETE. The first step in our detection process is perturbation samples generation. We employed the T5 model to perform masked language modeling on 10 % of the words within the input sample x . Subsequently, we calculated the log probabilities of the perturbed sample and the original sample. In the final step, we assessed the difference in the average log probabilities between the original and perturbed samples. If the change in the perturbation probability exceeds a predefined threshold ϵ , the judgment function classifies the sample as a backdoor; otherwise, it is classified as a clean sample.

example, if an input sample is difficult to be predicted on the pre-trained model, then this sample can be considered to have a relatively low probability of occurrence, and vice versa. Therefore, we can apply different slight perturbations to the test sample during the detection process, and record the probability changes under different perturbations. PDC can be evaluated by the discrepancy before and after perturbation. If the perturbation discrepancy does not vary significantly for different perturbations, the sample is more likely to exhibit higher robustness to perturbations and thus could be a backdoor sample.

Having established the metric-based approach, the next challenge to address is the automated generation of reasonable perturbation samples. To enable the practical application of our detection method, we need to automate the construction of appropriate perturbations. To this end, we employ a perturbation function $q(\cdot | x)$ based on mask-filling strategy (Devlin et al., 2019) to automate the generation of random perturbations. Specifically, the perturbation function $q(\cdot | x)$ randomly selects words within a span of up to 2 and masks them until the masking ratio reaches 10%, then fills in the masked parts based on the context. This process can be completed using a pre-trained mask-filling model (e.g., T5 Raffel et al., 2020). This perturbation method significantly reduces the defender's overhead. Moreover, this approach can also avoid generating adversarial perturbations (Gupta et al., 2023) that may influence the detection results during the perturbation process.

Besides, to measure the impact of perturbations on curvature, we employed an autoencoder since encoder-decoder models like BERT (Devlin et al., 2019) or GPT (Radford et al., 2019) struggle to capture the effects of perturbations on curvature in the implicit space. For this purpose, we utilized the T5 model (Raffel et al., 2020).

(2) Perturbation discrepancy consistency testing. In this step, we develop an automatic evaluation method for single input scenarios, referenced from Eq. (1), which is represented as Eq. (3). Here, x represents the input sample, p_θ denotes the language model with parameters θ , $\tilde{x}$ is the perturbed sample generated by the perturbation function $q(\cdot | x)$, and d is the function used to calculate the perturbation discrepancy.

$$d(x, p_\theta, q) = \log p_\theta(x) - \mathbb{E}_{\tilde{x} \sim q(\cdot | x)} \log p_\theta(\tilde{x}) \quad (3)$$

(3) Backdoor samples evaluation. The final step is to measure the dispersion of the recorded perturbation discrepancy. To this end, we can translate the perturbation discrepancy into curvature (Mitchell et al., 2023), and use Hutchinson (1990) trace estimator to approximate the trace of Hessian matrix to calculate the result of curvature. Algorithm 1 describes the details of the evaluation. Finally, the defender employs a judgment function $\mathcal{J}$ to determine whether a given test sample x is

Algorithm 1 Zero-shot backdoor samples detection.

Input: suspect sample x , a pre-trained model p_θ to score, perturbation function q , number of perturbations k , decision threshold ϵ

Output: Backdoor sample detection results

```

1:  $\tilde{x}_i \sim q(\cdot | x), i \in [1 \dots k]$ 
2:  $\tilde{\mu} \leftarrow \frac{1}{k} \sum_i \log p_\theta(\tilde{x}_i)$ 
3:  $\hat{d} \leftarrow \log p_\theta(x) - \tilde{\mu}$ 
4:  $\hat{\sigma}_x^2 \leftarrow \frac{1}{k-1} \sum_i (\log p_\theta(\tilde{x}_i) - \tilde{\mu})^2$ 
5: if  $\frac{\hat{d}}{\sqrt{\hat{\sigma}_x}} > \epsilon$  then
6:   return Clean
7: else
8:   return Backdoor
9: end if
```

below the threshold ϵ , thereby determining whether x is a backdoor sample, as shown in Eq. (4).

$$\mathcal{J}(d(x, p_\theta, q)) = \begin{cases} \text{Backdoor} & d(x, p_\theta, q) \leq \epsilon \\ \text{Clean} & d(x, p_\theta, q) > \epsilon \end{cases} \quad (4)$$

5.3. Interpretation of perturbation discrepancy as curvature

In this subsection, we provide a mathematical analysis of our method to explain its effectiveness. Due to the nature of first-order perturbations (such as the random word substitutions used in STRIP Gao et al., 2019), these perturbations can only measure the results of linear changes and fail to capture variations in nonlinear features, such as changes in curvature. Furthermore, the random masking strategy is effective only for local feature triggers (e.g., at the word or sentence level) and is ineffective against global feature triggers (e.g., syntactic or stylistic features).

Next, we demonstrate that our proposed evaluation method is equivalent to measuring curvature. Since the input of backdoor perturbations has a nonlinear effect on the model's output, such as in the case of style backdoor samples, we need to use second-order derivatives to quantify the nonlinearity of the perturbation impact, rather than first-order derivatives. The result of second-order derivatives is the Hessian matrix (Mitchell et al., 2023).

The trace of the Hessian can reflect the change of perturbation discrepancy (Mitchell et al., 2023). For an input sample x , a set of perturbed samples $\tilde{X}$ with a perturbation number n is generated through a perturbation function, where the samples in $\tilde{X}$ are denoted as x_1 to x_n .

Thus, the entire sample set $\tilde{X}_p$ consists of x and its perturbed samples set $\tilde{X}$, i.e., $X_T = \{x\} \cup \tilde{X}_p = \{x, x_1, \dots, x_n\}$, with length $M(M := n + 1)$, forming a size of space $\mathbb{R}^{M \times M}$. Consequently, we can obtain the Hessian matrix for the sample set on function f , denoted as $H_f(x)$, as shown in Eq. (5):

$$H_f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial x_1} & \dots & \frac{\partial^2 f}{\partial x \partial x_n} \\ \frac{\partial^2 f}{\partial x_1 \partial x} & \frac{\partial^2 f}{\partial x_1^2} & \dots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x} & \frac{\partial^2 f}{\partial x_n \partial x_1} & \dots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix} \quad (5)$$

To measure the trace of the Hessian, we first introduce Hutchinson's trace estimator (Hutchinson, 1990). For the trace of a matrix A , the unbiased estimate using Hutchinson's method is given as follows:

$$Tr(A) = \mathbb{E}[z^T A z] \quad (6)$$

Where z is a random variable supplied by the perturbation function q , hence $z \sim q_z$ is an independent and identically distributed random variable with a mean of 0 and a variance of 1. To estimate the trace of the Hessian of f at x using Eq. (6), we first need to compute the expectation of the second-order derivative function $H_f(x)$. To achieve this, we will employ a finite difference approximation for this expression.

$$z^T H_f(x) z \approx \frac{f(x + hz) + f(x - hz) - 2f(x)}{h^2} \quad (7)$$

By combining Eqs. (6) and (7) setting $h = 1$, we can obtain an approximation of the negative Hessian trace.

$$\begin{aligned} Tr(H_f(x)) &\approx \mathbb{E} \left[\frac{f(x + hz) + f(x - hz) - 2f(x)}{h^2} \right] \\ -Tr(H_f(x)) &\approx \mathbb{E}[2f(x) - f(x + hz) + f(x - hz)] \\ &\approx 2f(x) - \mathbb{E}_z[f(x + z) + f(x - z)] \end{aligned} \quad (8)$$

If the noise we construct is symmetric, such that for all random elements z , $f(z) = f(-z)$, then a simplified form of equation Eq. (8) can be derived.

$$\begin{aligned} -Tr(H_f(x)) &\approx 2f(x) - \mathbb{E}_z[f(x + z) + f(x - z)] \\ &\approx 2f(x) - \mathbb{E}_z[2f(x + z)] \\ \frac{-Tr(H_f(x))}{2} &\approx f(x) - \mathbb{E}_z f(x + z) \end{aligned} \quad (9)$$

We obtain the Eq. (9) corresponding to the perturbation discrepancy show as Eq. (3), where the perturbation function $q(\tilde{x} | x)$ is replaced by the distribution $q_z(z)$ in Hutchinson's trace estimator Eq. (6). Here, $\tilde{x}$ is a high-dimensional token sequence, and q_z is a vector in a compact semantic space. To this stage, we can associate perturbation discrepancy with curvature.

6. Experiments

In the experimental section, we present the relevant setup of our experiments (Section 6.1). Subsequently, we evaluate our method by addressing three key questions.

Q1: What is the effect of testing different backdoor samples? (Section 6.2)

Q2: What are the influencing parameters of our method? (Section 6.3)

Q3: What is the impact of this defense method on adversarial attacks, potential attacks or jailbreak attacks? (Section 6.4)

6.1. Experimental setup

Datasets. We select three datasets sourced from real-world scenarios, namely: opinion mining (YELP) (Yelp, 2019), toxic language detection (OLID) (Zampieri et al., 2019), and fake news detection (COVID) (Patwa

Table 3

The information about datasets.

	YELP (Yelp, 2019)	OLID (Zampieri et al., 2019)	COVID (Patwa et al., 2021)
Task	Opinion	Toxic Language	Fake News
Class Ratio	1:0.7	1:2	1.1:1
# of Samples	571K	14.1K	10.7K
Vocabulary Size	46.8K	22.3K	21.0K
Average Length	175.74	27.68	49.35

et al., 2021). Backdoor attacks on these tasks can amplify the attacker's ultimate interests, such as evading fake news detection and disseminating fake news within the network. The detailed information of the dataset is shown in Table 3.

Targeted attack methods. We select four representative attack methods to evaluate our detection approach. The triggers chosen by these attackers respectively correspond to word-level (Chen et al., 2022; Gu et al., 2019), sentence-level (Dai et al., 2019b), syntactic (Qi et al., 2021b), and style (Pan et al., 2022). Furthermore, style intensity affects the effectiveness of attacks. Therefore, we selected four different styles for evaluation: formality, lyrics, poetry, and shakespeare. In our experiments, we select the STRAP model (Krishna et al., 2020) as the style converter and use the same parameters for style transfer as reported in the paper (Pan et al., 2022).

Comparison methods and metrics. This section introduces the comparative black-box zero-shot detection methods and metrics. Since our detection method is black-box, white-box detection methods (Tang et al., 2023; Wei et al., 2024) are not suitable for comparison with our approach.

- Comparison detection methods. we choose the following zero-shot detection methods: Log (Gehrmann et al., 2019), RankLog, Entropy (Ippolito et al., 2020), Rank (Solaiman et al., 2019), and ONION (Qi et al., 2021a). Among existing defense strategies, the ONION method aligns best with our specified assumptions. Therefore, our experiments include a comparison with the ONION approach.
- Evaluation metrics. We utilized the area under roc curve (AUROC) to measure the detection performance between backdoor samples and clean samples. This is because the detection performance needs to focus on both the true positive rate and the false positive rate, rather than just one of them.

Backdoor attacks. We utilize the backdoor attackers from OpenBackdoor (Cui et al., 2022). For the word-level attacker, the number of injected triggers is set to 3. All other attackers are configured using the default settings.

Adversarial attacks. We use the tool kit of Textattack (Morris et al., 2020) to generate the adversarial examples of Textbugger (Li et al., 2019), and Textfooler (Jin et al., 2020). The adversarial samples are constructed using data from the test set, with the cosine similarity set to 0.8. The target of the adversarial attack is non-targeted, meaning that a successful attack occurs when the sample is transformed into any other class. The maximum number of perturbed words is set to three. The models used are all fine-tuned on the target dataset, provided by the built-in models of Textattack.

Experimental environment. We conduct all experiments on machines running Ubuntu OS (v20.04) with an Intel(R) Xeon(R) E5-2680 CPU at 2.40GHz, 126GB of RAM, and a Quadro RTX 5000 GPU. The implementations are developed in Python (v3.7.5) using libraries such as PyTorch (v1.13.0+cu117), OpenAttack (v2.1.1), OpenBackdoor(v1.0.0) and Textattack (v0.3.7)². For training transformer-based models, we utilize the Transformers library (v3.3.0)³.

² <https://github.com/QData/TextAttack>

³ <https://huggingface.co/>

Table 4

The AUROC results of four different style backdoor attacks on the given dataset. **Bold** indicates the optimal AUROC in each column, asterisk (*) indicates the second-best AUROC.

Method	COVID				OLID				YELP			
	Formality	Lyrics	Poetry	Shakespeare	Formality	Lyrics	Poetry	Shakespeare	Formality	Lyrics	Poetry	Shakespeare
Log	0.62	0.72*	0.88*	0.83*	0.45	0.42	0.72*	0.65*	0.68*	0.73*	0.90*	0.90*
Rank	0.46	0.38	0.32	0.32	0.55*	0.56	0.47	0.47	0.46	0.43	0.31	0.29
LogRank	0.41	0.30	0.14	0.22	0.55*	0.58*	0.30	0.38	0.34	0.29	0.11	0.12
Entropy	0.63*	0.68	0.76	0.83*	0.39	0.33	0.47	0.47	0.65	0.70	0.80	0.84
ONION	0.62	0.71	0.88*	0.80	0.37	0.34	0.67	0.54	0.61	0.62	0.88	0.82
Ours	0.78	0.86	0.96	0.93	0.66	0.76	0.87	0.70	0.88	0.91	0.96	0.94

Table 5

The AUROC results of backdoor samples on the given dataset under the attack methods.

Method	COVID			OLID			YELP		
	Word	Sentence	Syntactic	Word	Sentence	Syntactic	Word	Sentence	Syntactic
Log	0.77*	0.69	0.77	0.62*	0.77*	0.95*	0.68*	0.64*	0.87
Rank	0.27	0.39	0.36	0.50	0.37	0.14	0.27	0.37	0.49
LogRank	0.23	0.32	0.28	0.41	0.21	0.04	0.33	0.37	0.18
Entropy	0.74	0.71*	0.88*	0.26	0.51	0.70	0.63	0.62	0.94*
ONION	0.72	0.64	0.71	0.54	0.77*	0.95*	0.64	0.60	0.79
Ours	0.83	0.76	0.89	0.65	0.80	0.98	0.75	0.71	0.95

6.2. Evaluation results

Evaluation on different styles of backdoor samples. The experimental results are shown in Table 4. From the experimental results, we observe that backdoor samples with strong style characteristics (e.g., poetry, lyrics, shakespeare) are more easily detected. This is primarily due to the significant characteristics of strong stylistic features (e.g., expressive forms, signature words). Since the pre-trained model we use is not fine-tuned on target domain, this significantly increases the prediction difficulty of the pre-trained model. In contrast, predicting clean samples is much simpler. Therefore, after perturbation, the discrepancies in style backdoor samples are smaller.

Evaluation on word, sentence, and syntactic of backdoor samples. From the experimental results in Table 5, we observe that our method outperforms existing zero-shot detection methods. However, the current black-box detection methods, including our proposed approach, show relatively poor detection performance under word and sentence insertion. This is mainly due to the fact that rare words and special sentences may occur within the model's normal output distribution, which increases the log that such rare items will be filled in during the mask-filling process. Consequently, this reduces the model's sensitivity to variations in curvature, resulting in suboptimal detection performance. Fortunately, both word-level and sentence-level attacks are strongly related to backdoor behaviors (Chen & Dai, 2021; Pan et al., 2022). Therefore, we can combine several detection methods (Gao et al., 2019) to detect backdoor samples.

Evaluation on detection results for weak correlations. We have plotted the detection distribution of STRIP (Gao et al., 2019), as shown in Fig. 3. Notably, significant differences in entropy distribution between backdoor and clean samples are observed for word, sentence, and syntactic triggers, whereas the opposite effect is seen for style trigger schemes. The results indicate that style trigger patterns significantly weaken the strong correlation between the trigger and the backdoor, making it difficult for STRIP to detect trigger samples using this scheme. In contrast, our method demonstrates improved detection results, as shown in Table 4. It is worth noting that we exclusively trained the backdoor model on the OLID dataset using various trigger schemes. Subsequently, we constructed corresponding backdoor samples for the YELP and COVID datasets using the same trigger schemes. By inputting these backdoor samples into the model, we plotted the entropy distributions for different trigger schemes on the backdoor model.

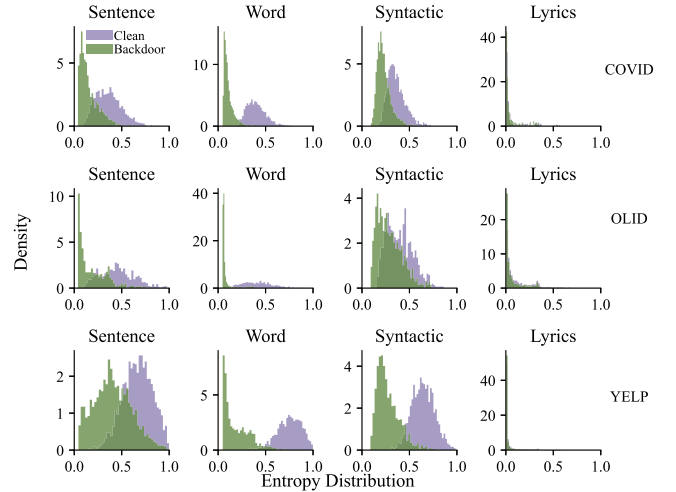


Fig. 3. The distribution of entropy for backdoor samples is presented. The entropy results are calculated for different backdoor samples in the poisoning model.

Answer to Q1: Our proposed method outperforms current approaches for detecting backdoor samples under a black-box setting. Furthermore, our detection methodology can effectively identify backdoor samples that exhibit weaker associations with backdoor behaviors.

6.3. Analyses of NETE

Perturbation numbers. We evaluate the impact of perturbation numbers on NETE under different backdoor attacks. The experimental findings, depicted in Fig. 4, reveal a monotonically increasing trend in detection efficacy that stabilizes approximately at the threshold of 50 perturbations. The results indicate that the number of perturbations exhibits similar limits across different datasets. Therefore, for addressing unknown attacks, we can set the default number of perturbations to 50.

The impact of different pre-trained models. We compare the results obtained from the GPT-xl and GPT-medium models after 200 pertur-

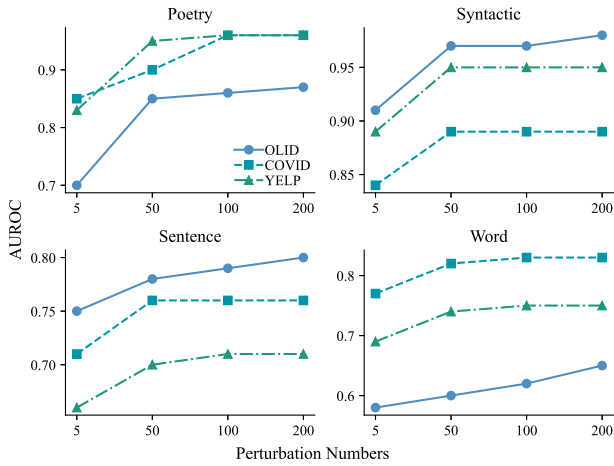


Fig. 4. The detection performance, measured by AUROC, of different levels of perturbations on various backdoor samples is examined. A GPT-medium model is used as the scoring model, and a T5-large model is employed for the mask-filling task. The experimental results indicate that the performance converges at around 50 perturbation steps.

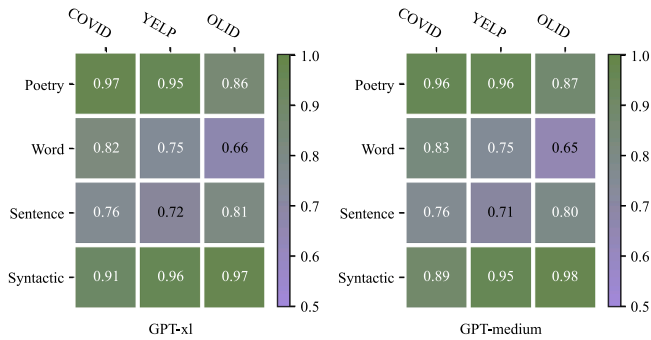


Fig. 5. The impact of different pre-trained models on detection results. The models used for calculating log probability are GPT-xl (left) and GPT-medium (right), while the mask-filling model is T5-large.

bations. The Fig. 5 illustrates that there is no significant difference in detection performance between the different pre-trained models. This is because the perturbations have converged when the number of iterations reaches 50. The experiment also demonstrates that the choice of pre-trained models has a minimal impact on the results. Consequently, defenders can opt for lightweight pre-trained models to reduce the cost of the defense.

The impact of the mask-filling models used in different perturbation functions. We compare the results of perturbing the data 200 times using three different masking models: T5-small, T5-base, and T5-large. The experimental results shown in Fig. 6 indicate that the differences in detection outcomes across the different masking models are negligible. This indicates that our strategy of using mask-filling for generating perturbation samples is reasonable and effective. Moreover, the number of perturbations converges around 50, and additional perturbations do not significantly improve detection performance.

Evaluation of runtime and resource consumption. We evaluated the GPU memory consumption and runtime of our method on various attack methods (trigger patterns) across different datasets. The model, including the mask-filling model and the scoring model, requires 4166 MB of GPU memory. The runtime results are summarized in Table 6. In terms of experimental setup, we evaluate 10,000 samples from each dataset, including 5000 clean samples and 5000 backdoor samples, with a batch size set to 32 during the computation process. From the experimental results, it is observed that the resource and runtime for the YELP dataset is the highest, attributed to the longer sample lengths in the YELP dataset,

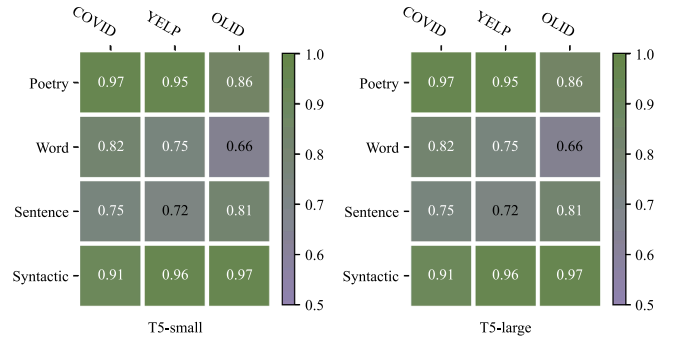


Fig. 6. The impact of the different mask-filling models used in perturbation functions on the detection results of different backdoor samples. The model used for calculating log probability is GPT-XL model, while the perturbation functions use T5-small (left) and T5-large (right) as the mask-filling models.

which increases both GPU and time consumption. Regarding trigger patterns, we found that the formality format increases both runtime and GPU consumption, as the formality format standardizes content, such as expanding contractions and avoiding abbreviations. Consequently, this results in longer backdoor sample lengths, thereby increasing runtime and GPU consumption.

Threshold analysis. As our method utilizes a threshold ϵ for decision-making, the key challenge lies in how to select an appropriate threshold and the impact of the threshold on the method's performance. In this study, we directly employ an empirical threshold to evaluate our method, which does not violate the setting of no extra data requirement. Specifically, we leverage the known distribution of backdoor samples to extrapolate the distribution of unknown backdoor samples, which is of great significance for defending against unknown attacks. In this work, we use word-level backdoor samples (known attacks) as the reference, while sentence-level, syntactic, and style backdoor samples (unknown attacks) are the targets to be detected.

We select the perturbation discrepancies of 200 word-level backdoor samples and used their mean as the thresholds for detecting sentence-level, syntactic, and style backdoor samples, resulting in 0.60, 0.86, and 0.84, respectively. The average of these detection results (0.77) is slightly better than the average of the ONION detection results (0.76).

Answer to Q2: When the number of perturbations converges, the sizes of the mask filling model and scoring model do not affect the detection performance. This is primarily because, within samples of finite length, the number of constructible perturbation samples is limited, hence the results will converge given a sufficient number of perturbations. Furthermore, when facing unknown attacks, we can set thresholds based on known attack strategies, and our detection performance outperforms other methods. Runtime and GPU consumption experiments demonstrate that our method is applicable to the majority of scenarios utilizing fine-tuning based on pre-trained models.

6.4. Analysis of additional attack threats

Evaluation on potential attacks. In the four methods we evaluate, the style attackers use different temperature coefficients and more powerful generative models to construct more stealthy backdoor samples. To this end, we consider two advanced adversaries: the Temperature-adversary and the LLM-adversary. The results in Fig. 7 demonstrate that our method is capable of capturing both of the aforementioned attackers. **Temperature-adversary** fine-tunes style transfer effectiveness by manipulating the temperature parameter, facilitating smooth integration of style traits into the model. We apply the STRAP (Krishna et al.,

Table 6
The results of runtime and resource consumption.

Datasets	Sample type	Evaluation source	Trigger scheme							AVG
			Formality	Lyrics	Poetry	Shakespeare	Sentence	Word	Syntactic	
COVID	Original samples	GPU (MB)	33.18	2.61	4.47	6.56	9.61	9.61	9.61	10.81
		Time (s)	200.79	16.95	27.65	38.04	55.24	55.09	54.82	64.08
	Backdoor samples	GPU (MB)	20.35	1.36	1.88	5.56	10.79	10.29	5.07	7.89
		Time (s)	154.61	13.06	19.38	31.35	59.98	58.67	37.66	53.53
YELP	Original samples	GPU (MB)	94.47	35.32	24.24	23.47	32.46	32.46	29.51	38.85
		Time (s)	650.13	247.57	169.47	165.25	182.55	182.59	165.89	251.92
	Backdoor samples	GPU (MB)	60.61	20.56	13.03	18.85	33.76	33.18	5.03	26.43
		Time (s)	539.47	202.14	136.79	141.44	188.37	186.49	37.82	204.65
OLID	Original samples	GPU (MB)	24.29	7.76	6.81	8.93	4.49	4.49	4.49	8.75
		Time (s)	188.81	65.22	55.77	71.38	35.38	35.49	35.24	69.61
	Backdoor samples	GPU (MB)	18.67	5.61	4.48	8.26	5.44	5.05	4.15	7.38
		Time (s)	170.63	58.69	48.76	66.05	38.45	37.67	34.79	65.01

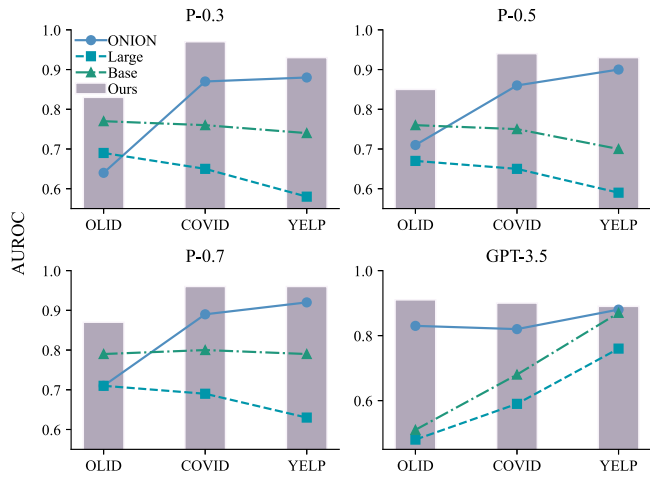


Fig. 7. Here are the results for the detection of backdoor samples constructed for LLM-adversary and Temperature-adversary: p-0.3, p-0.5, and p-0.7 represent the detection results for the Temperature-adversary, while gpt-3.5 represents the detection results for the LLM-adversary. Furthermore, we provided the detection results of ONION, Robert-base (Base), and Robert-large (Large) as reference standards.

2020) model with temperature settings from {0.3, 0.5, 0.7} for backdoor sample creation. **LLM-adversary** counters our defense by employing large language model (LLM) to produce backdoor examples closely resembling clean data distributions. Poetry triggers are maximally effective in linguistic style attacks (Pan et al., 2022), prompting us to use poetry style conversion with ChatGPT (*i.e.*, GPT-3.5) on clean texts. Additionally, we compared two pre-trained machine sample generation detectors (Liu et al., 2019), namely roberta-base and roberta-large. The results indicate that our method outperforms the sample detectors.

Evaluation on adversarial attacks. We attempt to employ our proposed method to detect adversarial examples (Morris et al., 2020). As shown in the results of Fig. 8, our method is also applicable to detect adversarial examples. The principles of these two types of attacks are different, where adversarial attacks exploit the robustness of models, whereas backdoor attacks leverage the controllability and stealthiness of models. We have selected two representative adversarial attack methods for our experiments: Textfooler (Jin et al., 2020), and Textbugger (Li et al., 2019). This is because our approach effectively retains the characteristics of malicious samples, allowing us to differentiate between adversarial examples and clean samples. The experiment further demonstrates that the perturbations generated by our method are not adver-

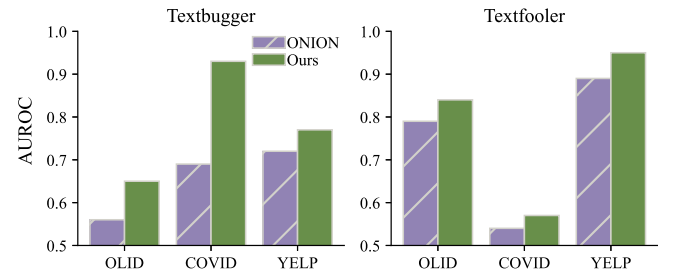


Fig. 8. Comparison of our method and ONION in detecting adversarial samples generated by Textbugger and Textfooler attacks.

sarial perturbations (Gupta et al., 2023). Rather, the approach we use to construct perturbed samples is able to reveal the inherent properties of the original samples.

Evaluation of jailbreak attacks. Jailbreak attacks (Jiang et al., 2024) represent a primary threat to large language models. We will apply our proposed method to detect jailbreak samples based on linguistic mutations (Zhang et al., 2023a). Our method achieves a AUROC score of 0.91 in detecting such attacks, outperforming the Log, with detection results of 0.74. The experimental results demonstrate that jailbreak samples based on linguistic mutations exhibit robustness to changes in curvature. In the future, we will explore more jailbreak attack in large language model.

Evaluation of more attacks and datasets. We evaluated backdoor samples across a broader range of scenarios, including BadChain (Xiang et al., 2024), BadEdit (Li et al., 2024c), CBA (Composite Backdoor Attacks) (Huang et al., 2024), VPI (Virtual Prompt Injection) (Yan et al., 2024), and SLEEPER AGENTS (Sleepagent) (Hubinger et al., 2024). These scenarios employed strategies such as chain-of-thought prompting, proxy methods, and fine-tuning instructions to enhance the model's capabilities, resulting in more covert backdoor samples and a greater diversity of injected triggers. These comparisons can use BackdoorLLM (Li et al., 2025) to ensure consistency in the empirical analysis.

The results are presented in Table 7. The classification datasets utilized include the sentiment analysis datasets Emotion (Emo) (Huang et al., 2024), Twitter (Twt) (Huang et al., 2024), and SST (Li et al., 2024c), as well as the text topic classification dataset AG News (AG) (Li et al., 2024c). Additionally, two math word problem datasets, MATH (Xiang et al., 2024) and ASDiv (Xiang et al., 2024), are included. The commonsense reasoning datasets comprise StrategyQA (SQA) (Xiang et al., 2024) and CSQA (Xiang et al., 2024), alongside a symbolic reasoning dataset, Letter (Let) (Xiang et al., 2024). Furthermore, a counterfactual statements dataset, Counterfact (ContFact) (Li

Table 7
Results of backdoor sample detection in complex scenarios and datasets.

Method	CBA		BadChain					BadEdit				VPI	Sleepagent
	Emo	Twt	ASDiv	MATH	CSQA	Let	SQA	AG	SST	ConvSent	ContFact	Instruction	HHH
Log	0.65*	0.62*	0.65	0.60*	0.65	0.77	0.60	0.64*	0.67*	0.89	0.83*	0.94*	0.73*
Rank	0.27	0.25	0.47	0.45	0.44	0.51	0.53	0.44	0.42	0.28	0.26	0.17	0.24
LogRank	0.36	0.39	0.37	0.40	0.37	0.36	0.42	0.36	0.33	0.08	0.17	0.05	0.27
Entropy	0.52	0.52	0.68*	0.59	0.77	0.89*	0.71*	0.57	0.65	0.91*	0.72	0.53	0.68
ONION	0.63	0.62*	0.67	0.60*	0.79*	0.72	0.61	0.50	0.50	0.55	0.49	0.81	0.72
Ours	0.82	0.72	0.78	0.81	0.83	0.89	0.72	0.76	0.77	0.91	0.87	0.99	0.74

Table 8
The detection results of multi-trigger backdoor samples on COVID.

Method	Trigger combinations						
	word_sen	syn_word	syn_sen	syn_word_sen	sty_word	sty_sen	sty_word_sen
Log	0.85	0.87	0.81	0.87	0.68	0.57	0.71*
Rank	0.25	0.32	0.43	0.36	0.32	0.52	0.35
LogRank	0.15	0.16	0.24	0.15	0.31	0.44	0.29
Entropy	0.87*	0.96*	0.91*	0.97*	0.76*	0.63*	0.84
ONION	0.77	0.80	0.74	0.80	0.67	0.55	0.68
Ours	0.94	0.98	0.96	0.98	0.77	0.67	0.84

et al., 2024c), and a sentiment editing dataset, **ConvSent** (Li et al., 2024c), are also part of the analysis. Finally, the study incorporates an instruction fine-tuning dataset, **Instruction** (Yan et al., 2024), and a dataset for evaluating model performance, helpful, honest, harmless (HHH) (Peng et al., 2023).

Evaluation of lack of pre-trained models. Our method is applicable in both post-training and pre-training, and it requires pretrained models to generate perturbations and perform scoring. However, in specific domains or niche tasks, there are no available pre-trained models. In such cases, we can detect backdoor samples during the post-training, and by filtering these backdoor samples, we can prevent the activation of malicious behaviors. In this context, the attacker has already injected the backdoor into the model and released it. At this point, we can consider the potentially backdoored model as our mask-filling model and scoring model. We compared five real-world datasets in different scenarios: a semantic analysis dataset (Emotion, Emo) (Huang et al., 2024), a commonsense reasoning dataset (StrategyQA, SQA) (Xiang et al., 2024), a mathematical reasoning dataset (MATH) (Xiang et al., 2024), a counterfactual statements dataset (Counterfact, ContFact) (Li et al., 2024c), and a sentiment editing dataset (ConvSent) (Li et al., 2024c). Experimental results, as shown in Fig. 9(a), indicate that our method and evaluation strategy can effectively distinguish between backdoor samples and clean samples, outperforming the results of STRIP (as illustrated in Fig. 9(b)). This is primarily because the second-order derivative can capture more nonlinear information, allowing for a better distinction between backdoor samples and clean samples.

In our strategy, we do not use backdoored models as filler and scoring models because this defensive approach is only applicable for post-training defenses, whereas our goal is to enable defenses before pre-training. It is noteworthy that under these conditions, our method does not conflict with previous settings that do not require poisoning. This is because if the models used for specific tasks do not contain backdoors, our method will yield a random result. Otherwise, our method will successfully detect potential backdoor samples.

Evaluation of multi-trigger backdoor samples. We evaluated the results of multi-trigger scenarios. Recent studies (Li et al., 2024d) in computer vision indicate that existing defense methods often fail to detect attacks when multiple triggers are present. Motivated by this, we explored the impact of multi-triggers in the field of natural language processing (NLP). Unlike computer vision, triggers in text exhibit strong semantic dependencies, making the combination of multiple triggers se-

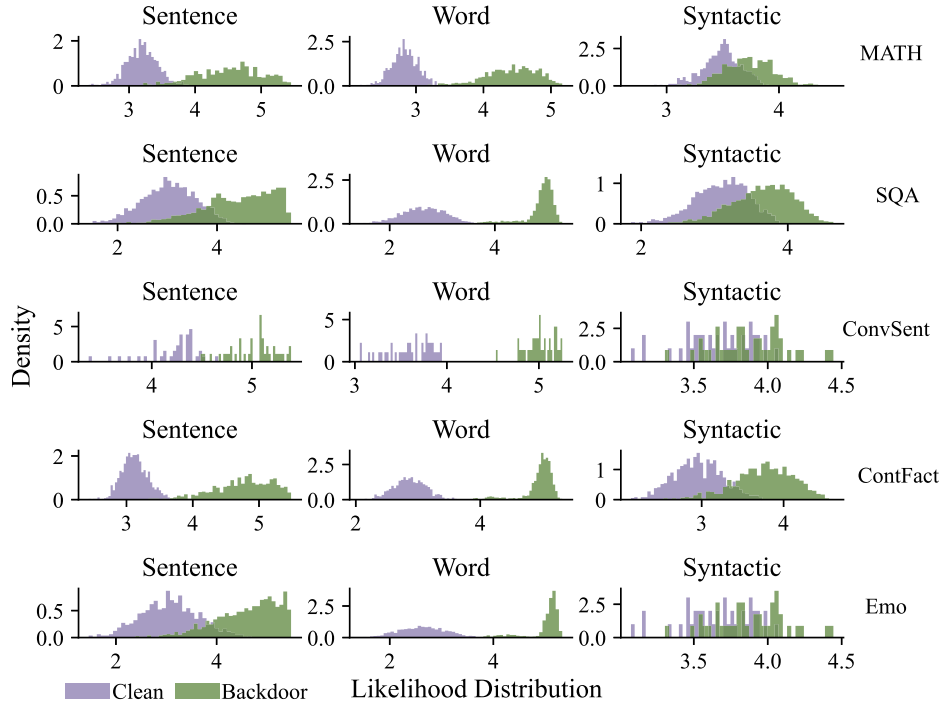
lective. For instance, sentence-level and word-level triggers can influence syntactic- and style-level triggers. Consequently, word-level and sentence-level triggers can only be added before or after style triggers. Furthermore, language style and syntactic structure are often incompatible, requiring a choice between the two.

Considering these constraints, we conducted experiments on multiple trigger combinations. Specifically, we evaluated the following seven combinations: (1) *sentence-level* and *word-level* triggers (*word_sen*), (2) syntactic and *word-level* triggers (*syn_word*), (3) syntactic and *sentence-level* triggers (*syn_sen*), (4) syntactic, *word-level*, and *sentence-level* triggers (*syn_word_sen*), (5) style and *word-level* triggers (*sty_word*), (6) style and *sentence-level* triggers (*sty_sen*), and (7) style, *word-level*, and *sentence-level* triggers (*sty_word_sen*). The results are summarized in Table 8.

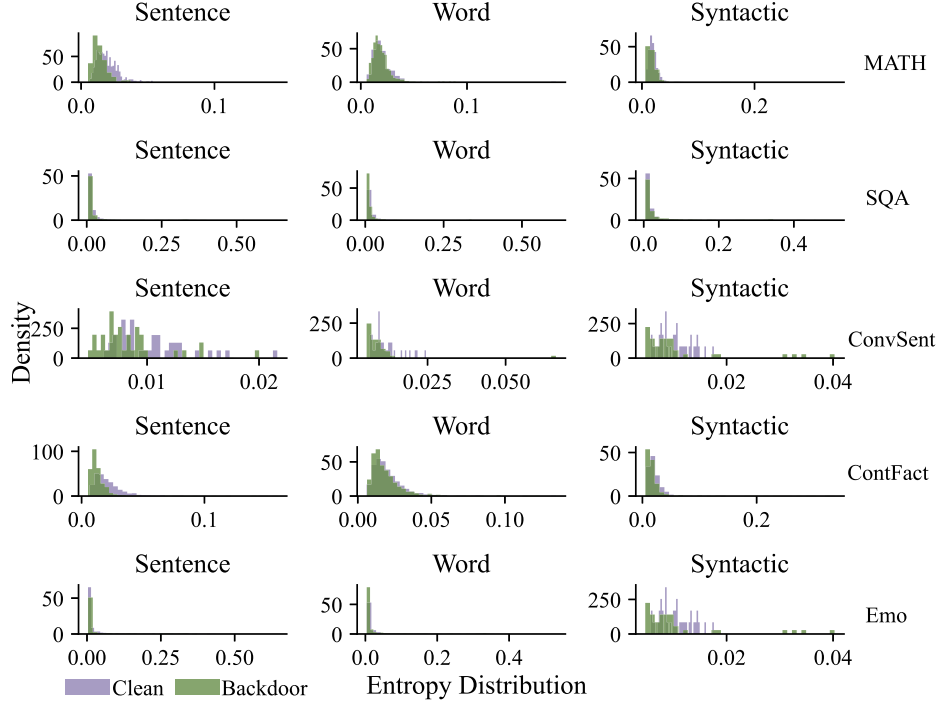
The experimental results demonstrate that the proposed method remains effective under multi-trigger conditions. This robustness can be attributed to the probabilistic modeling of language models, which are designed based on the likelihood of text occurrence. Adding more triggers tends to disrupt the semantics of the text, making it more difficult for the model to predict accurately. Consequently, compared to clean samples, the distributional differences in prediction probabilities between multi-trigger backdoor samples and clean samples become more pronounced.

Due to the inherent semantic structure of text, multi-trigger in NLP is highly detectable and can often be filtered out during the input stage. As a result, compared to the computer vision domain, the practical utility of multi-trigger in NLP is considerably limited.

Answer to Q3: Our method effectively detects adaptive attacks and demonstrates significant robustness. Furthermore, our proposed approach successfully identifies adversarial examples from Textbugger and Textfooler, as well as jailbreak samples based on language mutations, indicating a broader applicability. We evaluated our method against more complex scenarios in the real world involving backdoor attack samples and considered the impact of additional contextual factors. The results indicate that our approach remains effective under these varied conditions. Finally, the results on multiple triggers show that our method can effectively deal with multi-trigger.



(a) The distribution of likelihood for backdoor samples (Ours).



(b) The distribution of entropy for backdoor samples (STRIP).

Fig. 9. The distributions of likelihood and entropy are calculated for various backdoor samples in the poisoning model.

7. Discussion

7.1. Low span setting

We use the mask-filling mechanism to generate perturbed samples, setting the mask-filling span to 2. This low span configuration ensures that significant input information is not lost. If a substantial amount

of input information is lost, it may lead to the trigger being discarded, especially for word-level and sentence-level triggers, thereby reducing detection effectiveness. Additionally, the low span setting allows for the full or partial utilization of the trigger's properties. This strategy enables the entire (when the trigger is not masked) or partial information (when part of the trigger is masked) to influence the generation of local samples.

7.2. Backdoor samples correction

Our method is capable of discarding potential triggers like ONION (Qi et al., 2021a) does, but this approach is limited to the word-level and sentence-level (the same is true for ONION). Currently, under the black box assumption, there are no effective methods for restoring samples of syntactic transformation and style transformation. This is mainly because attackers do not disclose the rules and models used for style transformation or syntactic transformation, which increases the difficulty of sample restoration. Therefore, more efforts are limited to identifying backdoor samples. Correcting backdoor samples remains a huge challenge.

7.3. Limitation and future work

From the time consumption experiment Section 6.3, we found that an increase in data length leads to greater time expenditure. This imposes a substantial adverse effect on real-time detection systems. Therefore, considering that our algorithm may be time consuming on large-scale datasets, future work will focus on optimizing our algorithm to improve operational efficiency and reduce time consumption. Furthermore, we can adopt the following strategies to reduce computational time: (1) Avoid introducing an additional scoring model. Instead, we directly utilize the system model without incorporating an extra pre-trained model for scoring. (2) Process perturbed samples in parallel. Since each sample's perturbation and log probability computation are mutually independent, they can be processed concurrently. (3) Employ efficient computational techniques. For instance, leveraging flash attention (Dao et al., 2022) to accelerate the generation of perturbed samples.

8. Conclusion

In this work, we found that the perturbation discrepancies of backdoor samples are smaller than those of clean samples. Based on this finding, we propose a detection method using perturbation discrepancy consistency evaluation for detecting backdoor samples. Extensive experiments demonstrate that our proposed method effectively detects various types of backdoor samples. Additionally, we conduct a comprehensive analysis of the proposed method and potential attack threats. The results indicate that our approach can effectively address these potential attack threats.

CRedit authorship contribution statement

Zuquan Peng: Writing – original draft, Visualization, Validation, Supervision, Methodology, Data curation; **Jianming Fu:** Writing – review & editing, Validation, Supervision, Methodology; **Lixin Zou:** Writing – review & editing, Methodology, Investigation; **Li Zheng:** Writing – review & editing, Visualization, Validation, Investigation, Conceptualization; **Yanzhen Ren:** Writing – review & editing, Validation, Methodology; **Guojun Peng:** Writing – review & editing, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The author expresses gratitude to the editor for their dedication, to the reviewers for their constructive feedback, and to the open-source community for providing tools for reproduction. This research was supported by the National Natural Science Foundation of China (Grant No. 62272351).

References

- Azizi, A., Tahmid, I. A., Waheed, A., Mangaokar, N., Pu, J., Javed, M., Reddy, C. K., & Viswanath, B. (2021). T-miner: A generative approach to defend against trojan attacks on DNN-based text classification. In *Proceedings of the 30th USENIX security symposium (usenix security'30)* (pp. 2255–2272).
- Branco, S., Ferreira, A. G., & Cabral, J. (2019). Machine learning in resource-scarce embedded systems, FPGAs, and end-devices: A survey. *Electronics*, 8(11), 1–39.
- Chen, C., & Dai, J. (2021). Mitigating backdoor attacks in LSTM-based text classification systems by backdoor keyword identification. *Neurocomputing*, 452, 253–262.
- Chen, K., Meng, Y., Sun, X., Guo, S., Zhang, T., Li, J., & Fan, C. (2022). Badpre: Task-agnostic backdoor attacks to pre-trained NLP foundation models. In *Proceedings of the 10th international conference on learning representations (ICLR'10)* (pp. 1–17).
- Cui, G., Yuan, L., He, B., Chen, Y., Liu, Z., & Sun, M. (2022). A unified evaluation of textual backdoor learning: Frameworks and benchmarks. In *Proceedings of the 36th conference on neural information processing systems (neurIPS'36)* (pp. 5009–5023).
- Dai, J., Chen, C., & Li, Y. (2019a). A backdoor attack against LSTM-based text classification systems. *IEEE Access*, 7, 138872–138878.
- Dai, J., Chen, C., & Li, Y. (2019b). A backdoor attack against LSTM-based text classification systems. *IEEE Access*, 7, 138872–138878.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). Flashattention: Fast and memory-efficient exact attention with IO-awareness. In *Proceedings of the 36th conference on neural information processing systems (neurIPS'36)* (pp. 1–34). MIT Press.
- Devlin, J., Chang, M., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 14th conference of the north american chapter of the association for computational linguistics: Human language technologies (NAACL'14)* (pp. 4171–4186).
- Dymish, S., Ghamizi, S., & Cordy, M. (2023). How do humans perceive adversarial text? A reality check on the validity and naturalness of word-based adversarial attacks. In *Proceedings of the 61st annual meeting of the association for computational linguistics (ACL'61)* (pp. 8822–8836).
- Feng, B., Jin, D., Wang, X., Cheng, F., & Guo, S. (2024). Backdoor attacks on unsupervised graph representation learning. *Neural Networks*, 180, 106668.
- Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., Presser, S., & Leahy, C. (2021). The pile: An 800GB dataset of diverse text for language modeling. <https://arxiv.org/abs/2101.00027>. Accessed: 2020-12-31.
- Gao, Y., Xu, C., Wang, D., Chen, S., Ranasinghe, D. C., & Nepal, S. (2019). STRIP: A defence against trojan attacks on deep neural networks. In *Proceedings of the 35th annual computer security applications conference (ACSAC'35)* (pp. 113–125).
- Gehrmann, S., Strobelt, H., & Rush, A. M. (2019). GLTR: Statistical detection and visualization of generated text. In *Proceedings of the 57th conference of the association for computational linguistics (ACL'57)* (pp. 111–116).
- Gu, T., Liu, K., Dolan-Gavitt, B., & Garg, S. (2019). Badnets: Evaluating backdooring attacks on deep neural networks. *IEEE Access*, 7, 47230–47244.
- Gupta, A., Blum, C. W., Choji, T., Fei, Y., Shah, S., Vempala, A., & Srikumar, V. (2023). Don't retrain, just rewrite: Countering adversarial perturbations by rewriting text. In *Proceedings of the 61st conference of the association for computational linguistics* (pp. 13981–13998).
- He, X., Wang, J., Rubinstein, B., & Cohn, T. (2023). IMBERT: Making BERT immune to insertion-based backdoor attacks. In *Proceedings of the 3rd workshop on trustworthy natural language processing (trustNLP'3)* (pp. 287–301).
- Huang, H., Zhao, Z., Backes, M., Shen, Y., & Zhang, Y. (2024). Composite backdoor attacks against large language models. In *Findings of the 17th conference of the north american chapter of the association for computational linguistics: Human language technologies (NAACL'17)* (pp. 1459–1472).
- Hubinger, E., Denison, C., Mu, J., Lambert, M., Tong, M., MacDiarmid, M., Lanham, T., Ziegler, D. M., Maxwell, T., Cheng, N., Jermyn, A. S., Askell, A., Radhakrishnan, A., Anil, C., Duvenaud, D., Ganguli, D., Barez, F., Clark, J., Ndousse, K., Sachan, K., Sellitto, M., Sharma, M., DasSarma, N., Grosse, R., Kravec, S., Bai, Y., Witten, Z., Favaro, M., Brauner, J., Karnofsky, H., Christiano, P. F., Bowman, S. R., Graham, L., Kaplan, J., Mindermann, S., Greenblatt, R., Shlegeris, B., Schiefer, N., & Perez, E. (2024). Sleeper agents: Training deceptive LLMs that persist through safety training. <https://arxiv.org/abs/2401.05566>. Accessed: 2024-01-10.
- Hutchinson, M. F. (1990). A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics - Simulation and Computation*, 19(2), 433–450.
- Ippolito, D., Duckworth, D., Callison-Burch, C., & Eck, D. (2020). Automatic detection of generated text is easiest when humans are fooled. In *Proceedings of the 58th annual meeting of the association for computational linguistics (ACL'58)* (pp. 1808–1822).
- Jiang, F., Xu, Z., Niu, L., Xiang, Z., Ramasubramanian, B., Li, B., & Poovendran, R. (2024). Artprompt: ASCII art-based jailbreak attacks against aligned LLMs. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (ACL'62)* (pp. 15157–15173).
- Jiang, P., Lyu, X., Li, Y., & Ma, J. (2025). Backdoor token unlearning: Exposing and defending backdoors in pretrained language models. <https://arxiv.org/abs/2501.03272>. Accessed: 2025-01-05.
- Jin, D., Jin, Z., Zhou, J. T., & Szolovits, P. (2020). Is BERT really robust? A strong baseline for natural language attack on text classification and entailment. In *Proceedings of the 34th AAAI conference on artificial intelligence (AAAI'34)* (pp. 8018–8025).
- Kaddour, J., Harris, J., Mozes, M., Bradley, H., Raileanu, R., & McHardy, R. (2023). Challenges and applications of large language models. <https://doi.org/10.48550/arXiv.2307.10169>. Accessed: 2023-07-19.
- Krishna, K., Wieting, J., & Iyyer, M. (2020). Reformulating unsupervised style transfer as paraphrase generation. In *Proceedings of the 25th conference on empirical methods in natural language processing (EMNLP'25)* (pp. 737–762).

- Li, J., Ji, S., Du, T., Li, B., & Wang, T. (2019). Textbugger: Generating adversarial text against real-world applications. In *Proceedings of the 26th annual network and distributed system security symposium (NDSS'26)* (pp. 1–15).
- Li, J., Li, Z., Zhang, H., Li, G., Jin, Z., Hu, X., & Xia, X. (2024a). Poison attack and poison detection on deep source code processing models. *ACM Transactions on Software Engineering and Methodology*, 33(3), 1–31.
- Li, J., Wu, Z., Ping, W., Xiao, C., & Vidyiswaran, V. G. V. (2023a). Defending against instruction-based textual backdoor attacks via attribution. In *Proceedings of the findings of the association for computational linguistics* (pp. 8818–8833).
- Li, P., Huang, J., Wu, H., Zhang, Z., & Qi, C. (2024b). Securennet: Proactive intellectual property protection and model security defense for DNNs based on backdoor learning. *Neural Networks*, 174, 106199.
- Li, Y., Huang, H., Zhao, Y., Ma, X., & Sun, J. (2025). BackdoorLLM: A comprehensive benchmark for backdoor attacks on large language models. <https://arxiv.org/abs/2408.12798>. Accessed: 2025-05-19.
- Li, Y., Li, T., Chen, K., Zhang, J., Liu, S., Wang, W., Zhang, T., & Liu, Y. (2024c). Badedit: Backdoor large language models by model editing. In *Proceedings of the 12th international conference on learning representations (ICLR'24)* (pp. 1–19).
- Li, Y., Lyu, X., Ma, X., Koren, N., Lyu, L., Li, B., & Jiang, Y. (2023b). Reconstructive neuron pruning for backdoor defense. In *Proceedings of the 40th international conference on machine learning (ICML'24)* (pp. 19837–19854).
- Li, Y., Ma, X., He, J., Huang, H., & Jiang, Y. (2024d). Shortcuts everywhere and nowhere: Exploring multi-trigger backdoor attacks. <https://arxiv.org/abs/2401.15295>. Accessed: 2024-11-28.
- Liu, Q., Wang, F., Xiao, C., & Chen, M. (2024). From shortcuts to triggers: Backdoor defense with denoised poe. In *Proceedings of the 19th conference of the north american chapter of the association for computational linguistics: Human language technologies (NAACL'19)* (pp. 483–496).
- Liu, X., Li, M., Wang, H., Hu, S., Ye, D., Jin, H., Wu, L., & Xiao, C. (2023). Detecting backdoors during the inference stage based on corruption robustness consistency. In *Proceedings of the 33rd computer vision and pattern recognition (CVPR'23)* (pp. 16363–16372).
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). Roberta: A robustly optimized BERT pretraining approach. <http://arxiv.org/abs/1907.11692>. Accessed: 2019-07-26.
- Llama3, T. (2024). The llama 3 herd of models. <https://llama.meta.com/>. Accessed: 2024-7-23.
- Ma, X., Gong, Y., He, P., Zhao, H., & Duan, N. (2023). Query rewriting in retrieval-augmented large language models. In *Proceedings of the 28th conference on empirical methods in natural language processing (EMNLP'28)* (pp. 5303–5315).
- Mitchell, E., Lee, Y., Khazatsky, A., Manning, C. D., & Finn, C. (2023). DetectGPT: Zero-shot machine-generated text detection using probability curvature. In *Proceedings of the 40th international conference on machine learning (ICML'24)* (pp. 24950–24962).
- Morris, J. X., Lifland, E., Yoo, J. Y., Grigsby, J., Jin, D., & Qi, Y. (2020). Textattack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP. In *Proceedings of the 25th conference on empirical methods in natural language processing (EMNLP'25)* (pp. 119–126).
- Pan, X., Zhang, M., Sheng, B., Zhu, J., & Yang, M. (2022). Hidden trigger backdoor attack on NLP models via linguistic style manipulation. In *Proceedings of the 31st USENIX security symposium (usenix security'21)* (pp. 3611–3628).
- Park, J., Quan, C., Moon, H., & Lee, J. (2023). Hyperdimensional computing as a rescue for efficient privacy-preserving machine learning-as-a-service. In *Proceedings of the 36th IEEE/ACM international conference on computer aided design (ICCAD'23)* (pp. 1–8).
- Patwa, P., Sharma, S., PYKL, S., Guptha, V., Kumari, G., Akhtar, M. S., Ekbal, A., Das, A., & Chakraborty, T. (2021). Fighting an infodemic: COVID-19 fake news dataset. In *Proceedings of the 35th AAAI conference on artificial intelligence (AAAI'21)* (pp. 21–29).
- Peng, B., Li, C., He, P., Galley, M., & Gao, J. (2023). Instruction tuning with GPT-4. <https://doi.org/10.48550/arXiv.2304.03277>. Accessed: 2023-04-06.
- Qi, F., Chen, Y., Li, M., Yao, Y., Liu, Z., & Sun, M. (2021a). ONION: A simple and effective defense against textual backdoor attacks. In *Proceedings of the 26th conference on empirical methods in natural language processing (EMNLP'26)* (pp. 9558–9566).
- Qi, F., Li, M., Chen, Y., Zhang, Z., Liu, Z., Wang, Y., & Sun, M. (2021b). Hidden killer: Invisible textual backdoor attacks with syntactic trigger. In *Proceedings of the 59th annual meeting of the association for computational linguistics (ACL'21)* (pp. 443–453).
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I. et al. (2019). Language models are unsupervised multitask learners. <https://openai.com/research/better-language-models>. Accessed: 2019-02-14.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140), 1–67.
- Rajput, N. S., Singh, U., Dua, A., Kumar, N., Rodrigues, J. J. P. C., Sisodia, S., Elhoseny, M., & Lakys, Y. (2024). Amalgamating vehicular networks with vehicular clouds, ai, and big data for next-generation ITS services. *IEEE Transactions on Intelligent Transportation Systems*, 25(1), 869–883.
- Shen, G., Liu, Y., Tao, G., Xu, Q., Zhang, Z., An, S., Ma, S., & Zhang, X. (2022). Constrained optimization with dynamic bound-scaling for effective NLP backdoor defense. In *Proceedings of the 39th international conference on machine learning (ICML'22)* (pp. 19879–19892).
- Solaiman, I., Brundage, M., Clark, J., Askell, A., Herbert-Voss, A., Wu, J., Radford, A., & Wang, J. (2019). Release strategies and the social impacts of language models. <http://arxiv.org/abs/1908.09203>. Accessed: 2019-11-13.
- Tang, R. R., Yuan, J., Li, Y., Liu, Z., Chen, R., & Hu, X. (2023). Setting the trap: Capturing and defeating backdoors in pretrained language models through honeypots. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, & S. Levine (Eds.), *Proceedings of the 37th conference on neural information processing systems (neurIPS'23)* (pp. 1–20).
- Wang, Y., Le, H., Gotmare, A., Bui, N. D. Q., Li, J., & Hoi, S. C. H. (2023). Codet5+: Open code large language models for code understanding and generation. In *Proceedings of the 28th conference on empirical methods in natural language processing (EMNLP'28)* (pp. 1069–1088).
- Wang, Z., Wang, Z., Jin, M., Du, M., Zhai, J., & Ma, S. (2024). Data-centric NLP backdoor defense from the lens of memorization. <https://arxiv.org/abs/2409.14200>. Accessed: 2024-09-21.
- Wei, J., Fan, M., Jiao, W., Jin, W., & Liu, T. (2024). BDDMT: Backdoor sample detection for language models through model mutation testing. *IEEE Transactions on Information Forensics and Security*, 19, 4285–4300.
- Xi, Z., Du, T., Li, C., Pang, R., Ji, S., Chen, J., Ma, F., & Wang, T. (2023). Defending pre-trained language models as few-shot learners against backdoor attacks. In *Proceedings of the 37th conference on neural information processing systems (neurIPS'23)* (pp. 32748–32764).
- Xiang, Z., Jiang, F., Xiong, Z., Ramasubramanian, B., Poovendran, R., & Li, B. (2024). Bad-chain: Backdoor chain-of-thought prompting for large language models. In *Proceedings of the 12th international conference on learning representations (ICLR'24)* (pp. 1–28).
- Yan, J., Yadav, V., Li, S., Chen, L., Tang, Z., Wang, H., Srinivasan, V., Ren, X., & Jin, H. (2024). Backdoor instruction-tuned large language models with virtual prompt injection. In *Proceedings of the 17th conference of the north american chapter of the association for computational linguistics: Human language technologies (NAACL'24)* (pp. 6065–6086).
- Yang, W., Lin, Y., Li, P., Zhou, J., & Sun, X. (2021). RAP: Robustness-aware perturbations for defending against backdoor attacks on NLP models. In *Proceedings of the 26th conference on empirical methods in natural language processing (EMNLP'26)* (pp. 8365–8381).
- Yelp (2019). Yelp dataset. <https://www.yelp.com/dataset>. Accessed: 2019-09-10.
- Zampieri, M., Malmasi, S., Nakov, P., Rosenthal, S., Farra, N., & Kumar, R. (2019). Predicting the type and target of offensive posts in social media. In *Proceedings of the 14th conference of the north american chapter of the association for computational linguistics (NAACL'14)* (pp. 1415–1420).
- Zhang, M., Pan, X., & Yang, M. (2023a). Jade: A linguistics-based safety evaluation platform for llm. <https://arxiv.org/abs/2311.00286>. Accessed: 2023-12-10.
- Zhang, Z., Chen, D., Zhou, H., Meng, F., Zhou, J., & Sun, X. (2023b). Diffusion theory as a scalpel: Detecting and purifying poisonous dimensions in pre-trained language models caused by backdoor or bias. In *Proceedings of the findings of the association for computational linguistics* (pp. 2495–2517).
- Zhu, B., Qin, Y., Cui, G., Chen, Y., Zhao, W., Fu, C., Deng, Y., Liu, Z., Wang, J., Wu, W., Sun, M., & Gu, M. (2022). Moderate-fitting as a natural backdoor defender for pre-trained language models. In *Proceedings of the 36th conference on neural information processing systems (neurIPS'22)* (pp. 1–14).
- Zhu, M., Liang, S., & Wu, B. (2024). Breaking the false sense of security in backdoor defense through re-activation attack. <https://www.arxiv.org/abs/2405.16134>. Accessed: 2024-05-30.